Shielded Payments for Tangle Blueprints: Anonymous Pay-Per-Use Cloud Services via Cross-Chain Shielded Pools and Prepaid Credits

Drew Stone
drew@tangle.tools
Tangle Network

March 2026

Abstract

We present a privacy-preserving payment system for Tangle's Blueprint service protocol that enables anonymous pay-per-use access to AI inference, image generation, and general cloud compute. The system composes three layers: (1) a cross-chain UTXO-based shielded pool [1] with variable-amount JoinSplit transactions and multi-stablecoin wrapping, (2) a stateless gateway contract that bridges anonymous partial withdrawals to Tangle's service lifecycle, and (3) a prepaid credit scheme where a single zero-knowledge proof funds an account from which arbitrarily many jobs are authorized via EIP-712 signatures. Unspent funds remain shielded as change UTXOs, each withdrawal is unlinkable to prior withdrawals or the original deposit, and operators learn nothing about the payer's on-chain identity. The construction requires no modifications to Tangle's audited core contracts.

1 Introduction

Every major AI inference provider—OpenAI, Anthropic, Google—requires account registration, identity verification, and payment methods traceable to the user. When a developer queries Claude or GPT-4 for code review, the provider knows who asked, what was asked, and when. Decentralized alternatives inherit the same problem: on-chain payments link wallet addresses to service consumption, creating a permanent public record of every API call.

Buterin and Crapis [2] proposed "ZK API Usage Credits" for anonymous inference: deposit into a smart contract, generate a zero-knowledge proof per API call to authorize payment without revealing identity. The cryptographic foundation is sound, but a Groth16 proof per API call is impractical—$\sim$640ms generation time and $\sim$300k gas on-chain per request.

We present a system where:

1. A user deposits stablecoins into a shielded pool (public, one-time).
2. The user partially withdraws via a single ZK proof to fund a pseudonymous credit account. The remainder stays shielded as a change UTXO, unlinkable to the withdrawal.
3. The user authorizes many job payments from the credit account via cheap ECDSA signatures ($\sim$50k gas each), with no further ZK proofs required.
4. When the credit account is depleted, the user generates a new ZK proof for another partial withdrawal—each withdrawal is unlinkable to the previous ones.

The operator serves requests and claims payment, but never learns the user's wallet address, deposit history, or total balance. On-chain observers see tokens flow through the shielded pool but cannot connect depositors to spenders.

1.1 Contributions

- A UTXO-based privacy model where partial withdrawals produce unlinkable change notes, enabling incremental spending from a single deposit without linkability across withdrawal sessions.
- A two-phase payment scheme: one ZK proof amortized over many ECDSA-signed job payments, reducing per-job cost from ∼300k gas to ∼50k gas.
- A gateway architecture bridging VAnchor shielded pools to Tangle's service lifecycle with zero modifications to audited core contracts.
- Multi-asset wrapping (USDC, USDT, DAI → single pool token) to maximize the anonymity set across stablecoin ecosystems.
- A cross-chain bridge via LayerZero V2 enabling deposits on one chain and spending on another.

2 Preliminaries

We adopt notation from the Webb Protocol [1]. Let $H : \mathbb{F}_p^* \to \mathbb{F}_p$ denote Poseidon, a zkSNARK-friendly collision-resistant hash [6]. Let $(\mathsf{pk}, \mathsf{sk})$ denote a keypair where $\mathsf{pk} = H(\mathsf{sk})$.

Definition 1 (UTXO Commitment)**.** A commitment to a UTXO with chain identifier i, value v, public key pk, and blinding factor $r \xleftarrow{\$} \mathbb{F}_p$ is:

$$\mathsf{cm} = H(i, v, \mathsf{pk}, r)$$

Definition 2 (Nullifier)**.** The nullifier for commitment cm at Merkle path index j with signature $\sigma = H(\mathsf{sk}, j, \mathsf{cm})$ is:

$$\mathsf{sn} = H(\mathsf{cm}, j, \sigma)$$

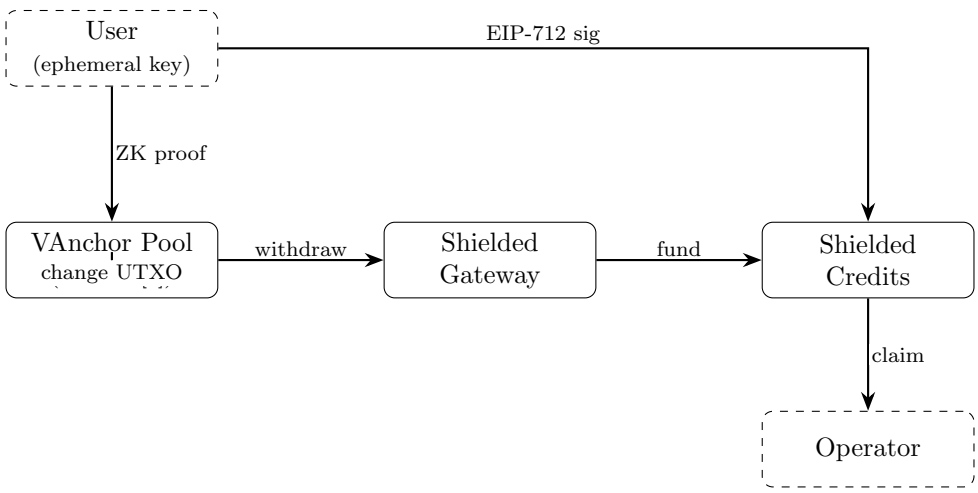
The signature σ binds the nullifier to the secret key, preventing nullifier grinding.

Definition 3 (Cross-Chain Merkle Membership)**.** Given Merkle roots $\mathbf{MR} = \{\mathsf{MR}_1, \ldots, \mathsf{MR}_k\}$ from k connected anchors, commitment cm satisfies membership if:

$$\mathsf{reconstruct}(\mathsf{cm}, \mathsf{path}(\mathsf{cm}), H) \in \mathbf{MR}$$

Empty edge slots (unconnected chains) are filled with a deterministic zero hash.

3 System Architecture



3.1 UTXO Model and Partial Withdrawals

The VAnchor implements a UTXO-based shielded pool with JoinSplit transactions (2-in-2-out or 16-in-2-out). Unlike fixed-denomination mixers, each UTXO carries an arbitrary value. A partial withdrawal works as follows:

1. User holds $\mathsf{UTXO}(v)$ in the pool (e.g., $v = 100$ USDC).
2. User generates a ZK proof spending $\mathsf{UTXO}(100)$ with:
   - Output 1: withdrawal of 10 to the gateway ($\mathsf{extAmount} = -10$)
   - Output 2: new $\mathsf{UTXO}(90)$ re-shielded in the pool (change note)
3. The circuit enforces $\mathsf{public_amount} + \sum v_{\mathsf{in}} = \sum v_{\mathsf{out}}$, i.e., $-10 + 100 = 90$.
4. The original UTXO's nullifier is marked spent. The change UTXO receives a fresh commitment with fresh blinding—**unlinkable to the original**.

The user can repeat this process: withdraw 10 more from $\mathsf{UTXO}(90)$, producing $\mathsf{UTXO}(80)$, and so on. Each withdrawal uses fresh nullifiers and commitments. An observer sees withdrawals from the pool but cannot determine whether they originate from the same depositor.

3.2 Shielded Gateway

The `ShieldedGateway` is a stateless proxy. It executes a VAnchor withdrawal (validating the zkSNARK proof and marking nullifiers), receives tokens, and atomically forwards them to:

- `shieldedRequestService`: Creates a Tangle service with the gateway as owner and ephemeral addresses as permitted callers.
- `shieldedFundService`: Tops up an existing service's escrow.
- `shieldedFundCredits`: Funds a prepaid credit account (Section 3.3).

Property 1 (Atomicity)**.** The gateway holds no funds beyond a single transaction. If any downstream call fails, the entire transaction—including the VAnchor withdrawal—reverts.

Property 2 (Non-Linkability)**.** The gateway stores no mapping between nullifiers and service/credit identifiers. The on-chain trace shows `VAnchor` → `Gateway` → `Credits` but reveals nothing about which depositor initiated the flow.

3.3 Shielded Credits

A prepaid credit system that amortizes one ZK proof over many service invocations.

Definition 4 (Credit Account)**.** Identified by $\mathsf{C} = \mathsf{keccak256}(\mathsf{pk}_s \| \mathsf{salt})$ where pk_s is an ephemeral ECDSA public key. State: $(\mathsf{pk}_s, \mathsf{token}, \mathsf{balance}, \mathsf{nonce})$.

Definition 5 (Spend Authorization)**.** An EIP-712 [5] typed signature:

$$\mathsf{SpendAuth} = \mathsf{Sign}_{\mathsf{sk}_s}(\mathsf{C}, \mathsf{serviceId}, \mathsf{jobIndex}, \mathsf{amount}, \mathsf{operator}, \mathsf{nonce}, \mathsf{expiry})$$

The designated $\mathsf{operator}$ address is the only entity that can claim the payment.

The credit lifecycle: FundCredits pulls tokens from the gateway and initializes the account. AuthorizeSpend verifies the EIP-712 signature, deducts balance, increments nonce, and stores a pending claim. ClaimPayment transfers tokens to the designated operator. Only $\mathsf{msg.sender} = \mathsf{operator}$ can claim.

4 Privacy Analysis

4.1 What the Operator Learns

Information	Operator learns?	Chain observer learns?
User's wallet address	No	No
User's deposit amount	No	No
User's total shielded balance	No	No
Which deposit funded this credit	No	No
Credit commitment C	Yes	Yes
Amount paid per job	Yes	Yes
Request content (prompt, image, etc.)	Yes	No
Linkage between jobs from same C	Yes	Yes
Linkage between different Cs	No	No

The operator knows *what* was requested and *how much* was paid, but not *who* is paying. This is analogous to paying cash at a store—the merchant sees the transaction but cannot trace it to a bank account.

4.2 Unlinkability Across Sessions

When a credit account C_1 is depleted, the user generates a new partial withdrawal from their shielded UTXO, producing a fresh commitment C_2 with a new ephemeral key. The two credit accounts are cryptographically unlinkable:

- Different ephemeral keys $\mathsf{pk}_{s_1} \neq \mathsf{pk}_{s_2}$
- Different commitments $\mathsf{C}_1 \neq \mathsf{C}_2$
- Different nullifiers consumed in each withdrawal proof
- The change UTXO connecting them has a fresh commitment with fresh blinding

An observer who sees both credit accounts cannot determine if they belong to the same user. The anonymity set is bounded by the total number of depositors in the pool.

4.3 Payment Security

Property 3 (No Double-Spend)**.** Balance deducted atomically before authorization is stored. Strict nonce ordering prevents replay.

Property 4 (No Front-Running)**.** Each spend authorization names a specific operator. Only that address can call ClaimPayment.

Property 5 (Key Immutability)**.** Once set, the spending key cannot be changed, preventing re-keying attacks on discovered commitments.

4.4 Limitations

Intra-session linkability. Multiple spends from the same credit account C are linkable. This is inherent to the prepaid model—full per-job unlinkability requires a ZK proof per job.

Operator sees request content. Payment privacy does not imply computation privacy. The operator processes the user's prompt/image/query. TEE-based execution (future work) would address this.

Anonymity set size. Privacy degrades if few users deposit into the pool. Multi-asset wrapping (USDC + USDT + DAI into one pool) and cross-chain deposits help grow the set.

No IP-level privacy. Without Tor or a mixnet, the user's network identity is visible to the operator. Orthogonal to this work.

5 AI Service Use Cases

The system is designed for any Blueprint-based cloud service but is primarily motivated by AI workloads where usage patterns are sensitive.

5.1 LLM Inference

A developer querying an LLM for code review, legal analysis, or medical information may not want the inference provider to associate those queries with their identity. With shielded payments:

- The developer deposits USDC into the pool from their known wallet.
- They withdraw to a credit account and submit prompts to a vLLM-based inference Blueprint.
- The operator serves the inference and claims payment, but cannot link the queries to the developer's wallet, ENS name, or on-chain history.
- When the credit is depleted, the developer creates a new credit account from their shielded balance—the new session is unlinkable to the previous one.

5.2 Image and Video Generation

Creative professionals generating images via Stable Diffusion or video via Sora-class models may wish to keep their prompts and creative direction private. The same flow applies: deposit once, generate many images across multiple unlinkable credit sessions.

5.3 AI Agent Execution

Autonomous AI agents operating on-chain need to pay for inference, tool use, and compute without revealing their controller's identity. An agent deposits into the shielded pool, funds credits, and pays for services autonomously. The agent's on-chain identity (a smart contract or EOA) is decoupled from its controller.

5.4 Fine-Tuning and Training

Users submitting proprietary datasets for fine-tuning can pay anonymously, preventing competitors from monitoring which models are being trained and by whom. The operator receives the data and payment but cannot attribute the training job to a specific organization.

5.5 General Cloud Compute

Beyond AI, any Blueprint-based service—databases, storage, CI/CD, web hosting—can accept shielded payments. The credit system is service-agnostic; it simply authorizes token transfers from a pseudonymous account to a designated operator.

6 Cross-Chain Architecture

The VAnchor supports cross-chain deposits via connected Merkle roots. A user deposits USDC on Ethereum, and the Merkle root is relayed to Base via LayerZero V2. The user then generates a withdrawal proof on Base, citing the Ethereum root in the cross-chain membership proof. The circuit validates membership against any of the k connected roots.

The system deploys with $\mathsf{maxEdges} = 7$ (supporting 8 chains total). The circuit is compiled for 8-root membership proofs; unoccupied edge slots are filled with deterministic zero hashes. Chains are added incrementally via the `addEdge` function on the LayerZero bridge contract—no redeployment required.

Initial deployment targets: Ethereum, Arbitrum, Base, Hyperliquid, BSC.

7 Cost Analysis

Approach	Per-job proof	Per-job gas	Per-job latency
ZK proof per call [2]	Groth16	∼300k	∼640ms
Direct VAnchor withdrawal	Groth16	∼300k	∼640ms
Shielded Credits (this work)	**ECDSA sig**	**∼50k**	**<100ms**
x402 [4] (no privacy)	None	∼30k	<50ms

The one-time ZK proof cost (∼640ms, ∼300k gas) is amortized over all jobs in a credit session. A user who funds $50 in credits and pays $0.10 per inference call amortizes the proof cost over 500 calls.

8 Implementation

Available on branch `feat/shielded-payments` of `tnt-core`. 47 Forge tests (including full E2E lifecycle) and 40 TypeScript SDK tests (including real Groth16 proof generation against 8-edge circuits).

References

[1] D. Stone, "Webb Protocol: A cross-chain private application and governance protocol," *IACR ePrint 2023/260*, 2023.

[2] V. Buterin and D. Crapis, "ZK API Usage Credits," presented at ETHMumbai, March 2026.

[3] Veridise Inc., "Security audit of Webb Protocol Solidity contracts," 2023.

[4] Coinbase, "x402: An open standard for internet-native payments," 2025.

[5] R. Floersch and B. Johnson, "EIP-712: Typed structured data hashing and signing," Ethereum Improvement Proposals, 2017.

[6] L. Grassi *et al.*, "Poseidon: A new hash function for zero-knowledge proof systems," *USENIX Security*, 2021.

[7] J. Groth, "On the size of pairing-based non-interactive arguments," *EUROCRYPT*, 2016.